Karamanoğlu Mehmetbey Üniversitesi
Mühendislik Fakültesi
Bilgisayar Mühendisliği Bölümü

Büyük Veriye Giriş Dersi
Vize Projesi

Apache Spark MLlib ve Rastgele Orman Algoritması ile Banka Müşteri Kaybı (Churn) Tahmin Analizi

Dr. Öğr. Üyesi Hüseyin ELDEM
250801007 - İrem ATIŞ

İÇERİK

1. Giriş ve Problem Tanımı
2. Kullanılan Teknolojiler ve Veri Seti
3. Uygulama Aşamaları
4. Veri Ön İşleme (Preprocessing)
5. Yöntem ve Algoritma
6. Bulgular ve Model Performansı
7. Değişken Önem Sıralaması (Feature Importance)
8. Literatür Karşılaştırması
9. Sonuç ve Öneriler
10. Kaynaklar

1. GİRİŞ VE PROBLEM TANIMI

- Müşteri kaybı mevcut müşterilerin bir kurumla olan ticari ilişkisini sonlandırmasıdır.
- Bankacılık sektöründe yeni bir müşteri kazanmanın maliyeti, mevcut bir müşteriyi elde tutma maliyetinden **5 ila 25 kat** daha fazladır.
- **Çözüm Yaklaşımı:** Veri madenciliği ve makine öğrenmesi teknikleri ile "ayrılma eğilimi" olan müşterileri henüz bankayı terk etmeden tespit etmek.
- **Projenin Amacı:** 10.000 kişilik bir veri setinden anlamlı paternler çıkararak, bankanın kayıplarını minimize edecek bir erken uyarı sistemi tasarlamak.

2. KULLANILAN TEKNOLOJİLER VE VERİ SETİ

- **Apache Spark (PySpark):** Büyük veri setlerini bellek içi (in-memory) işleme yeteneği sayesinde klasik kütüphanelerden çok daha hızlı sonuç üretir.
- **Spark MLlib:** Ölçeklenebilir makine öğrenmesi algoritmaları için optimize edilmiş kütüphane.
- **Veri Seti Kaynağı:** Kaggle - Bank Customer Churn Dataset.
- **Veri Boyutu:** 10,000 Kayıt, 12 Öznitelik (Feature).
- Müşteri ID, Ülke, Yaş, Kredi skoru, Soyad…
- **Hedef Değişken (Label):** Exited (1: Ayrıldı, 0: Kaldı).

3. UYGULAMA AŞAMALARI:

```
!pip install pyspark

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("BankaMusteriAnalizi").getOrCreate()
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
```

Bu kod bloğunda, PySpark kütüphanesi kullanılarak Apache Spark ortamı hazırlanmıştır. İlk olarak gerekli kütüphane yüklenmiş, ardından Spark ile çalışmak için temel bileşen olan SparkSession içe aktarılmıştır. Son olarak “BankaMusteriAnalizi” isimli bir Spark oturumu oluşturularak büyük veri işlemlerinin gerçekleştirileceği çalışma ortamı başlatılmıştır.

```python
dosya_yolu = "/content/sample_data/Bank Customer Churn Prediction.csv"


df = spark.read.csv(dosya_yolu, header=True, inferSchema=True)

df.printSchema()
df.show(5)
```

```
root
 |-- customer_id: integer (nullable = true)
 |-- credit_score: integer (nullable = true)
 |-- country: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- age: integer (nullable = true)
 |-- tenure: integer (nullable = true)
 |-- balance: double (nullable = true)
 |-- products_number: integer (nullable = true)
 |-- credit_card: integer (nullable = true)
 |-- active_member: integer (nullable = true)
 |-- estimated_salary: double (nullable = true)
 |-- churn: integer (nullable = true)

+-----------+------------+-------+------+---+------+---------+---------------+-----------+-------------+----------------+-----+
|customer_id|credit_score|country|gender|age|tenure|  balance|products_number|credit_card|active_member|estimated_salary|churn|
+-----------+------------+-------+------+---+------+---------+---------------+-----------+-------------+----------------+-----+
|   15634602|         619| France|Female| 42|     2|      0.0|              1|          1|            1|       101348.88|    1|
|   15647311|         608|  Spain|Female| 41|     1| 83807.86|              1|          0|            1|       112542.58|    0|
|   15619304|         502| France|Female| 42|     8| 159660.8|              3|          1|            0|       113931.57|    1|
|   15701354|         699| France|Female| 39|     1|      0.0|              2|          0|            0|        93826.63|    0|
|   15737888|         850|  Spain|Female| 43|     2|125510.82|              1|          1|            1|         79084.1|    0|
+-----------+------------+-------+------+---+------+---------+---------------+-----------+-------------+----------------+-----+
only showing top 5 rows
```

Bu kod bloğunda, banka müşteri verilerini içeren CSV dosyası Spark ortamına yüklenmiştir. Belirtilen dosya yolu üzerinden veri seti spark.read.csv() fonksiyonu ile okunmuş, header=True parametresi ile sütun isimleri korunmuş ve inferSchema=True ile veri tipleri otomatik olarak belirlenmiştir. Ardından printSchema() komutu ile veri setinin yapısı (sütunlar ve veri tipleri) incelenmiş, show(5) ile de ilk 5 satır görüntülenerek veri setine genel bir bakış sağlanmıştır.

```python
from pyspark.ml.feature import StringIndexer, VectorAssembler

indexer = StringIndexer(inputCols=["country", "gender"], outputCols=["country_index", "gender_index"])
df_indexed = indexer.fit(df).transform(df)


required_columns = ["credit_score", "country_index", "gender_index", "age", "tenure",
                    "balance", "products_number", "credit_card", "active_member", "estimated_salary"]


assembler = VectorAssembler(inputCols=required_columns, outputCol="features")
final_data = assembler.transform(df_indexed)

final_data = final_data.select("features", "churn")
final_data.show(5)
```

```
+--------------------+-----+
|            features|churn|
+--------------------+-----+
|[619.0,0.0,1.0,42...|    1|
|[608.0,2.0,1.0,41...|    0|
|[502.0,0.0,1.0,42...|    1|
|[699.0,0.0,1.0,39...|    0|
|[850.0,2.0,1.0,43...|    0|
+--------------------+-----+
only showing top 5 rows
```

Bu kod bloğunda, makine öğrenmesi modeli için veri ön işleme adımları gerçekleştirilmiştir. Öncelikle kategorik değişkenler olan “ülke(country)” ve “cinsiyet(gender)”, metin indeksleme (StringIndexer) kullanılarak sayısal indekslere dönüştürülmüştür. Ardından modelde kullanılacak tüm bağımsız değişkenler belirlenmiş ve vektör birleştirici (VectorAssembler) ile bu değişkenler tek bir “özellikler (features)” vektörü altında birleştirilmiştir. Son olarak modelin kullanacağı “features” ve hedef değişken olan “churn” seçilmiş ve veri setinin ilk 5 satırı görüntülenerek işlem kontrol edilmiştir.

4. VERİ ÖN İŞLEME (PREPROCESSING)

- **Metin İndeksleme (StringIndexing):** "Country" (Ülke) ve "Gender" (Cinsiyet) gibi kategorik metin verileri, makinenin işleyebileceği sayısal indekslere dönüştürüldü.
- **Tekil-Sıcak Kodlama (OneHotEncoding):** Sayısal indeksler arasındaki yapay büyüklük-küçüklük ilişkisini önlemek için veriler ikili (binary) sisteme yayıldı.
- **Vektör Birleştirici (VectorAssembler):** Tüm bağımsız değişkenler (yaş, bakiye, kredi skoru vb.), Spark MLlib'in girdi olarak kabul ettiği tek bir **"Feature Vector"** yapısında birleştirildi.

```python
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.evaluation import MulticlassClassificationEvaluator


train_data, test_data = final_data.randomSplit([0.8, 0.2], seed=42)


rf = RandomForestClassifier(labelCol="churn", featuresCol="features", numTrees=10)

rf_model = rf.fit(train_data)


predictions = rf_model.transform(test_data)


predictions.select("prediction", "churn").show(10)
```

```
+----------+-----+
|prediction|churn|
+----------+-----+
|       0.0|    0|
|       0.0|    0|
|       1.0|    1|
|       0.0|    1|
|       0.0|    0|
|       1.0|    1|
|       0.0|    1|
|       0.0|    0|
|       0.0|    0|
|       0.0|    0|
+----------+-----+
only showing top 10 rows
```

Bu kod bloğunda, veri seti kullanılarak bir makine öğrenmesi modeli oluşturulmuş ve test edilmiştir. Öncelikle veri seti %80 eğitim ve %20 test olacak şekilde ikiye ayrılmıştır. Ardından Random Forest algoritması tanımlanarak model eğitim verisi üzerinde eğitilmiştir. Eğitilen model, test verisi üzerinde tahmin yapmak için kullanılmış ve son olarak modelin tahmin sonuçları ile gerçek değerler karşılaştırılarak ilk 10 sonuç görüntülenmiştir.

5. YÖNTEM VE ALGORİTMA (RANDOM FOREST)

- **Yöntem:** Rastgele Orman (Random Forest) Sınıflandırıcı
- **Pipeline (Boru Hattı) Mimarisi:** Veri ön işleme ve modelleme adımlarının tek bir iş akışında birleştirilmesi.
- **Topluluk Öğrenmesi (Ensemble Learning):** Birden fazla karar ağacı oluşturularak "çoğunluğun oyu" ile tahmin yapılması.
- **Veri Bölümleme:** Veri setinin **%80'i eğitim**, **%20'si test** amacıyla ayrılmıştır.

```python
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.mllib.evaluation import MulticlassMetrics


evaluator = MulticlassClassificationEvaluator(labelCol="churn", predictionCol="prediction", metricName="accuracy")
accuracy = evaluator.evaluate(predictions)


f1_evaluator = MulticlassClassificationEvaluator(labelCol="churn", predictionCol="prediction", metricName="f1")
f1_score = f1_evaluator.evaluate(predictions)

print(f"Modelin Doğruluk Oranı (Accuracy): {accuracy:.4f}")
print(f"Modelin F1 Skoru: {f1_score:.4f}")

predictionAndLabels = predictions.select("prediction", "churn").rdd.map(lambda x: (float(x[0]), float(x[1])))
metrics = MulticlassMetrics(predictionAndLabels)

print("\nHata Matrisi (Confusion Matrix):")
print(metrics.confusionMatrix().toArray())
```

```
Modelin Doğruluk Oranı (Accuracy): 0.8501
Modelin F1 Skoru: 0.8280
/usr/local/lib/python3.12/dist-packages/pyspark/sql/context.py:157: FutureWarning: Deprecated in 3.0.0. Use SparkSession.builder.getOrCreate() instead.
  warnings.warn(

Hata Matrisi (Confusion Matrix):
[[1482.   38.]
 [ 250.  151.]]
```

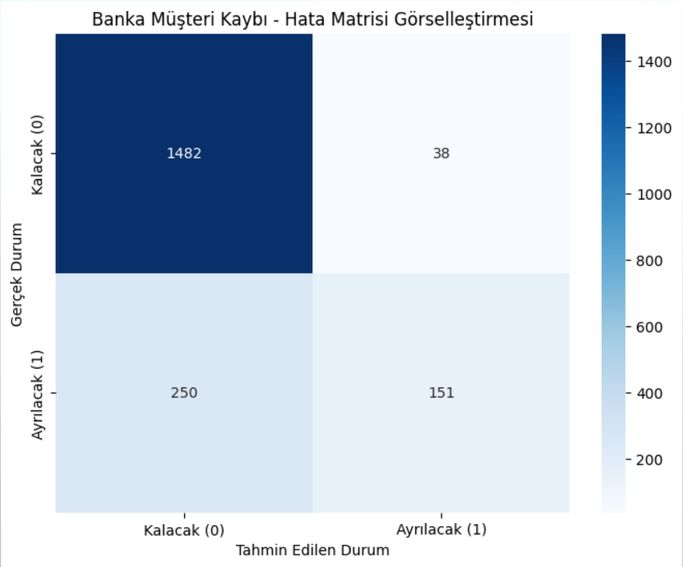
Bu kod bloğunda, eğitilen modelin performansı çeşitli değerlendirme metrikleri kullanılarak analiz edilmiştir. Öncelikle MulticlassClassificationEvaluator (Sınıflandırma Değerlendiricisi) ile modelin doğruluk (accuracy) ve F1 skoru hesaplanmıştır. Ardından tahmin ve gerçek değerler RDD formatına dönüştürülerek MulticlassMetrics kullanılmış ve modelin hata matrisi (confusion matrix) elde edilmiştir. Böylece modelin doğru ve yanlış tahminleri detaylı bir şekilde değerlendirilmiştir.

```python
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np


conf_matrix = np.array([[1482, 38], [250, 151]])


plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='g', cmap='Blues',
            xticklabels=['Kalacak (0)', 'Ayrılacak (1)'],
            yticklabels=['Kalacak (0)', 'Ayrılacak (1)'])

plt.xlabel('Tahmin Edilen Durum')
plt.ylabel('Gerçek Durum')
plt.title('Banka Müşteri Kaybı - Hata Matrisi Görselleştirmesi')
plt.show()
```



Bu kod bloğunda, modelden elde edilen hata matrisi görselleştirilmiştir. Öncelikle hata matrisi NumPy dizisi olarak tanımlanmış, ardından matplotlib ve seaborn kütüphaneleri kullanılarak ısı haritası (heatmap) şeklinde çizdirilmiştir. Grafik üzerinde eksenler “gerçek” ve “tahmin edilen” değerleri gösterecek şekilde etiketlenmiş ve başlık eklenmiştir. Böylece modelin doğru ve yanlış tahminleri görsel olarak daha anlaşılır hale getirilmiştir.

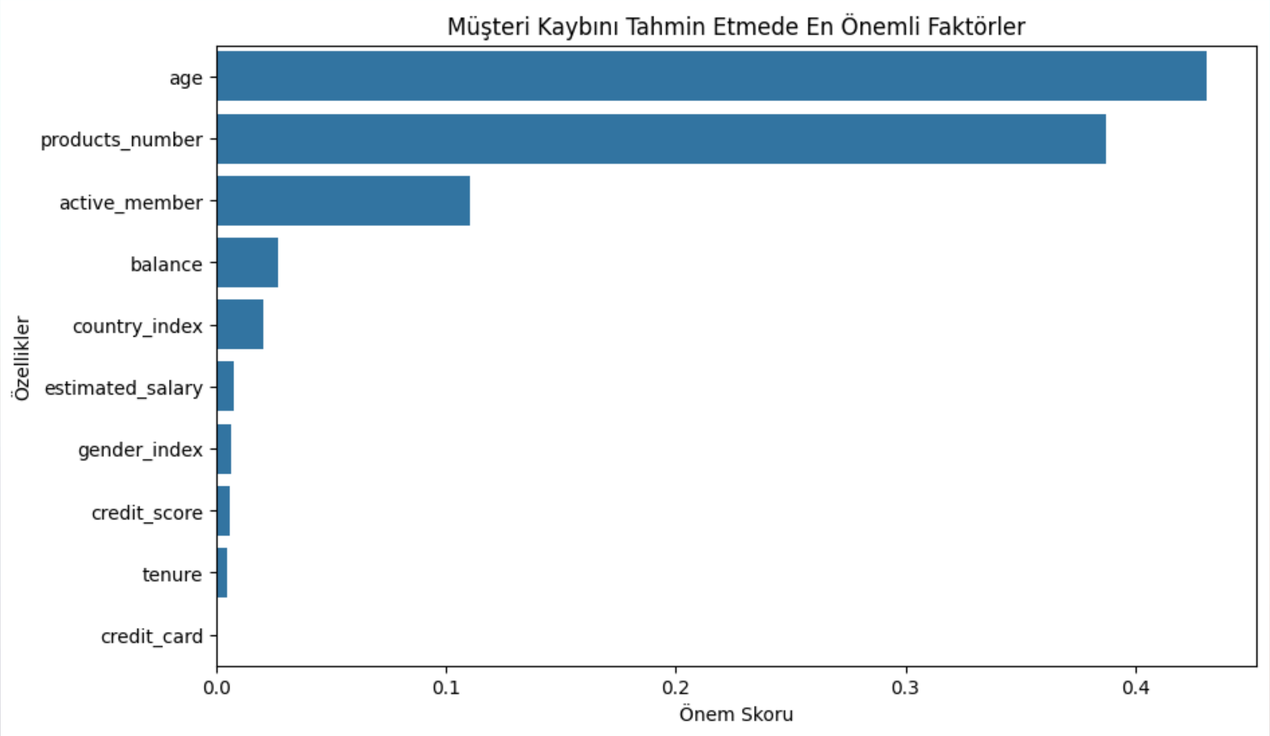
6. BULGULAR VE MODEL PERFORMANSI

- **Genel Doğruluk (Accuracy):** %85.01
- **F1-Skoru:** %82.80 (Hassasiyet ve Duyarlılığın dengeli bir göstergesi).
- **Hata Matrisi Analizi (Confusion Matrix):**
  - **Doğru Tahminler:** 1482 (Kalacak) + 151 (Gidecek) = **1633 başarılı tahmin.**
  - **Hatalı Tahminler:** 38 (Yanlış Alarm) + 250 (Kaçırılan Churn).
- **Model Yorumu:** Model, bankada kalacak müşterileri %97 gibi çok yüksek bir kesinlikle bilmektedir.

```python
import pandas as pd

importances = rf_model.featureImportances
feature_list = required_columns
feature_imp = pd.Series(importances.toArray(), index=feature_list).sort_values(ascending=False)

plt.figure(figsize=(10,6))
sns.barplot(x=feature_imp, y=feature_imp.index)
plt.title("Müşteri Kaybını Tahmin Etmede En Önemli Faktörler")
plt.xlabel("Önem Skoru")
plt.ylabel("Özellikler")
plt.show()
```



Bu kodda, eğitilmiş Random Forest modelinin hangi değişkenlere daha fazla önem verdiği feature importance yöntemi ile hesaplanmış ve değişkenlerle eşleştirilerek sıralanmıştır. Elde edilen önem değerleri görselleştirilerek, müşteri kaybını (churn) tahmin etmede en etkili değişkenler grafiksel olarak sunulmuştur.

7. DEĞİŞKEN ÖNEM SIRALAMASI (FEATURE IMPORTANCE)

- **Model Karar Mekanizması:** Random Forest algoritmasının karara varırken en çok hangi verilere güvendiği analiz edilmiştir.
- **En Önemli Faktörler:**
  1. **Yaş (Age):** En belirleyici kriter.
  2. **Ürün Sayısı (NumOfProducts):** Müşterinin bankayla bağını gösterir.
  3. **Bakiye (Balance):** Finansal sadakat göstergesi.
- **Düşük Etkili Faktörler:** Kredi kartı sahipliği ve Cinsiyet.

8. LİTERATÜR KARŞILAŞTIRMASI:

- **Yöntem:** Literatürde sıkça kullanılan Lojistik Regresyon ve Karar Ağaçlarına kıyasla **Random Forest** algoritması, "topluluk öğrenmesi" sayesinde daha yüksek kararlılık ve doğruluk sunmaktadır.
- **Teknoloji:** Çoğu çalışma yerel araçlarla (Scikit-learn) yapılırken, bu projede kullanılan **Apache Spark** altyapısı, sistemin büyük veri ölçeğinde çalışabilmesini sağlayarak literatürdeki geleneksel yaklaşımlardan ayrışmaktadır.
- **Performans:** Elde edilen **%85 başarı oranı**, bankacılık sektörü churn analizleri için literatürde "yüksek ve uygulanabilir" kabul edilen standartlarla tam uyumludur.
- Literatürdeki benzer çalışmalarla karşılaştırıldığında, Random Forest algoritmasının tekli modellere göre daha kararlı sonuçlar verdiği görülmüştür. Ayrıca çalışmamızda kullanılan Spark altyapısı, analizimizi geleneksel yöntemlerden ayırarak büyük veri setlerine ölçeklenebilir bir yapı sunmaktadır.

9. SONUÇ VE ÖNERİLER

- Apache Spark altyapısı ile saniyeler içinde 10.000 veri işlenmiş ve tahminleme yapılmıştır.
- **Model Başarısı:** Apache Spark altyapısı ve Random Forest algoritması ile **%85,01 doğruluk** oranına ulaşılmıştır. %85 başarı oranı, gerçek dünya bankacılık verileri için kabul edilebilir ve başarılı bir seviyedir.
- **Temel Bulgular:** Müşteri kaybında en belirleyici faktörlerin **yaş**, **ürün sayısı** ve **hesap bakiyesi** olduğu saptanmıştır.
- **İş Stratejisi:** Modelin "ayrılma riski yüksek" olarak belirlediği müşteri grubuna yönelik kişiselleştirilmiş sadakat kampanyaları uygulanmalıdır.
- **Öneri:** Veri setindeki dengesizliği gidermek için "Aşırı Örnekleme" (Oversampling) teknikleri denenebilir.

10. KAYNAKLAR

1. **Apache Spark Documentation.** "Machine Learning Library (MLlib) Guide." [online]. Erişim: spark.apache.org
2. **Karau, H., et al.** "Learning Spark: Lightning-Fast Big Data Analysis." O'Reilly Media. (Büyük veri işleme ve Spark mimarisi için).
3. **Breiman, L.** "Random Forests." Machine Learning Journal. (Rastgele Orman algoritmasının teorik temeli).
4. **Kaggle Dataset.** "Bank Customer Churn Dataset." [online]. Erişim: kaggle.com (Veri setinin kaynağı).
5. **Brownlee, J.** "Data Preparation for Machine Learning." (Veri ön işleme ve VectorAssembler yöntemleri için).